
Framework de desarrollo seguro basado en métodos computacionales guiados por conocimiento

Plan técnico de desarrollo, arquitectura y selección de herramientas
DevSecOps para arquitecturas de microservicios bajo el ciclo Design Science Research

Índice

1	Introducción	3
1.1	Propósito del documento	3
1.2	Cambios respecto a la versión 1.0	3
1.3	Alcance	3
1.4	Criterios de selección de herramientas	3
2	Principios de diseño	5
3	Arquitectura general	6
3.1	Las tres capas del framework	6
3.2	Vista de despliegue: dónde corre cada cosa	6
4	Stack tecnológico completo	8
4.1	Resumen del stack	8
4.2	Lenguajes de programación: dónde se usa cada uno	10
4.2.1	Por qué NestJS y no otro framework	10
4.2.2	Por qué Prisma	11
4.3	Interfaz del framework: React + Vite	11
5	Núcleo de conocimiento (capa superior)	12
5.1	Base de conocimiento	12
5.1.1	Qué contiene	12
5.1.2	Cómo se almacena: grafo de conocimiento en Neo4j	12
5.1.3	Por qué Neo4j	13
5.1.4	¿Cuál es más rápido?	13
5.2	Motor de razonamiento	14
5.2.1	Por qué OPA	15
5.3	Trazabilidad y retroalimentación	15
5.4	SARIF como formato común	16
6	Uso de modelos de lenguaje (LLM)	17
6.1	Principio de uso	17
6.2	Mapa de uso por fase	17
6.3	Selección del modelo	18
6.4	Adaptador de proveedor	18
6.5	Riesgos específicos del uso de LLM	19
7	Pipeline de construcción y validación (capa media)	20
7.1	Fase 1: Adquisición de requisitos	20
7.2	Fase 2: Análisis y propuesta arquitectónica	20
7.3	Fase 3: Generación de la estructura interna	21
7.4	Fase 4: Security Gate 1, análisis estático	22
7.5	Fases 5 y 6: Refinamiento iterativo guiado por conocimiento	23
7.5.1	Salvaguardas obligatorias	23
7.6	Fase 7: Integración de microservicios	24
7.7	Fase 8: Security Gate 2, seguridad de integración	25
7.8	Fase 9: Pruebas automatizadas	25
7.9	Fase 10: Contenerización	26
7.10	Fase 11: DAST y Security Gate 3	26

8 Infraestructura, CI/CD y notificaciones	28
8.1 GitHub Actions como orquestador del pipeline	28
8.2 Un paso por fase	28
8.3 Notificaciones en Telegram	29
8.3.1 Diseño	29
8.3.2 Buenas prácticas aplicadas	29
8.4 Entornos	30
9 Entradas y salidas del framework	31
9.1 Formas de entrada	31
9.2 Salida: el entregable	31
9.3 Informe de evidencia y trazabilidad	31
10 Seguridad del propio framework	32
11 Evaluación del artefacto (ciclo DSR)	33
11.1 Relación entre el ciclo DSR y el plan	33
11.2 Métricas	33
11.3 Diseño experimental	33
11.3.1 Conjuntos de requisitos	33
11.3.2 Líneas base de comparación	34
11.3.3 Calibración de las herramientas dinámicas	34
11.4 Dashboard de métricas	34
12 Estructura del repositorio del framework	35
13 Organización del equipo y hoja de ruta	36
13.1 Capacidad disponible	36
13.2 Reparto de responsabilidades	36
13.3 Resumen de sprints	36
13.4 Cobertura de las fases por sprint	38
13.5 Hitos de control	39
13.6 Cronograma diario	39
13.7 Prioridades y plan de contingencia	42
14 Riesgos del proyecto y mitigación	43
A Glosario	44

1. Introducción

1.1 Propósito del documento

El objetivo del proyecto es automatizar e integrar la seguridad (DevSecOps) y las buenas prácticas de diseño en todo el ciclo de vida del software para arquitecturas de microservicios, usando conocimiento formalizado y reglas computacionales en lugar de revisiones manuales aisladas.

Este documento es la guía de implementación. Para cada componente del framework indica qué herramienta se usa, para qué, por qué se eligió frente a sus alternativas y qué produce. Incluye además la planificación por sprints para un equipo de dos personas entre el 1 de octubre de 2026 y el 15 de enero de 2027. Está pensado para que el equipo pueda seguirlo de principio a fin sin decisiones tecnológicas pendientes.

1.2 Cambios respecto a la versión 1.0

Cambio	Impacto
Código generado en TypeScript	Los microservicios se generan siempre con NestJS y TypeScript. El motor del framework sigue en Python. Se ajustan plantillas, reglas SAST, pruebas, salvaguardas e imágenes.
Solo backend	El framework genera únicamente microservicios backend; no genera frontend.
Dashboard en React	La interfaz del propio framework se construye con React + Vite . No forma parte de la salida del framework.
Neo4j como base de conocimiento	Neo4j sustituye a PostgreSQL + pgvector: grafo de conocimiento, trazabilidad e índice vectorial nativo para RAG en una sola base.
Notificaciones en Telegram	Cada fase del pipeline informa su progreso en un chat de Telegram. El workflow de CI se divide en un paso por fase.
Planificación	Hoja de ruta ajustada al calendario real del equipo (62 días de trabajo, dos integrantes).

1.3 Alcance

El framework recibe un documento de requisitos en lenguaje natural y entrega un sistema de microservicios backend en NestJS, contenerizado, probado y validado en seguridad, acompañado de un informe de evidencia que traza cada requisito hasta el código y los controles que lo satisfacen.

- **Dentro del alcance:** generación de microservicios backend en NestJS/TypeScript, tres Security Gates automatizados, refinamiento iterativo guiado por conocimiento, integración con API Gateway, event bus e Identity Provider, pruebas automatizadas, análisis dinámico, trazabilidad completa y notificaciones del pipeline en Telegram.
- **Fuera del alcance (trabajo futuro):** generación de frontend, generación en otros lenguajes o frameworks, despliegue en Kubernetes, service mesh, despliegue a producción real.

1.4 Criterios de selección de herramientas

Cada herramienta se eligió aplicando los mismos cinco criterios, en este orden de prioridad:

1. **Beneficio para la pregunta de investigación:** que aporte evidencia medible al framework.
2. **Facilidad de adopción:** curva de aprendizaje baja, buena documentación, instalación con un comando o una imagen Docker.
3. **Integración en pipeline:** ejecutable por línea de comandos, salida en formato estructurado (idealmente SARIF o JSON) y tiempos de ejecución compatibles con CI.
4. **Licencia abierta y coste cero:** reproducibilidad académica sin depender de licencias comerciales.
5. **Madurez y comunidad:** proyecto activo, respaldado por una fundación u organización reconocida (OWASP, CNCF, etc.).

2. Principios de diseño

Estos principios guían todas las decisiones técnicas del documento. Cuando surja una duda durante la implementación, se resuelve volviendo a ellos.

P1. El LLM propone, las reglas deciden

Los modelos de lenguaje generan, extraen, corrigen y explican. Nunca aprueban un Security Gate ni toman una decisión arquitectónica final. Toda decisión que afecte la seguridad la toma un motor de reglas determinista (Open Policy Agent) a partir de la base de conocimiento, y queda registrada con la regla que la justifica. Así cada decisión es reproducible, auditable y explicable.

P2. Un formato común para todos los resultados

Todas las herramientas de análisis (SAST, SCA, secretos, contenedores, DAST) se normalizan al formato estándar SARIF. Esto permite comparar hallazgos entre iteraciones, detectar regresiones mediante *fingerprints* y alimentar la trazabilidad con una sola estructura de datos.

P3. Primero de punta a punta, después inteligente

La primera entrega es un *walking skeleton*: un microservicio NestJS escrito a mano que atraviesa los tres Security Gates en CI. Solo cuando la tubería funciona completa se añade la generación con LLM y el razonamiento. Esto evita descubrir problemas de integración al final del proyecto.

P4. Una tecnología de salida, bien resuelta

El motor del framework se escribe en Python y los microservicios generados se escriben siempre en NestJS con TypeScript, a partir de una única plantilla. Lo que varía entre ejecuciones es qué servicios se crean, qué patrones se aplican y qué controles se incluyen, no la tecnología. Así las plantillas, las reglas y las pruebas se escriben una sola vez y los experimentos son comparables. La extensibilidad a otros frameworks se diseña (plantillas y reglas intercambiables) pero no se implementa en esta versión.

P5. Conocimiento como código

La base de conocimiento vive en archivos versionados en Git (YAML, Rego y reglas Semgrep) y se carga en Neo4j al iniciar el motor. Cada cambio en el conocimiento tiene autor, fecha y motivo, y la evaluación puede reproducir exactamente con qué versión del conocimiento se obtuvo cada resultado.

3. Arquitectura general

3.1 Las tres capas del framework

El framework se organiza en tres capas, tal como el diagrama conceptual del proyecto:

- **Capa superior, núcleo de conocimiento (KBS):** base de conocimiento, motor de razonamiento y trazabilidad. Da soporte a todas las fases.
- **Capa media, pipeline de construcción y validación:** once fases desde los requisitos hasta los contenedores validados, con tres Security Gates y un ciclo de refinamiento.
- **Capa inferior, marco metodológico:** el ciclo Design Science Research, que organiza la construcción y la evaluación del propio framework.

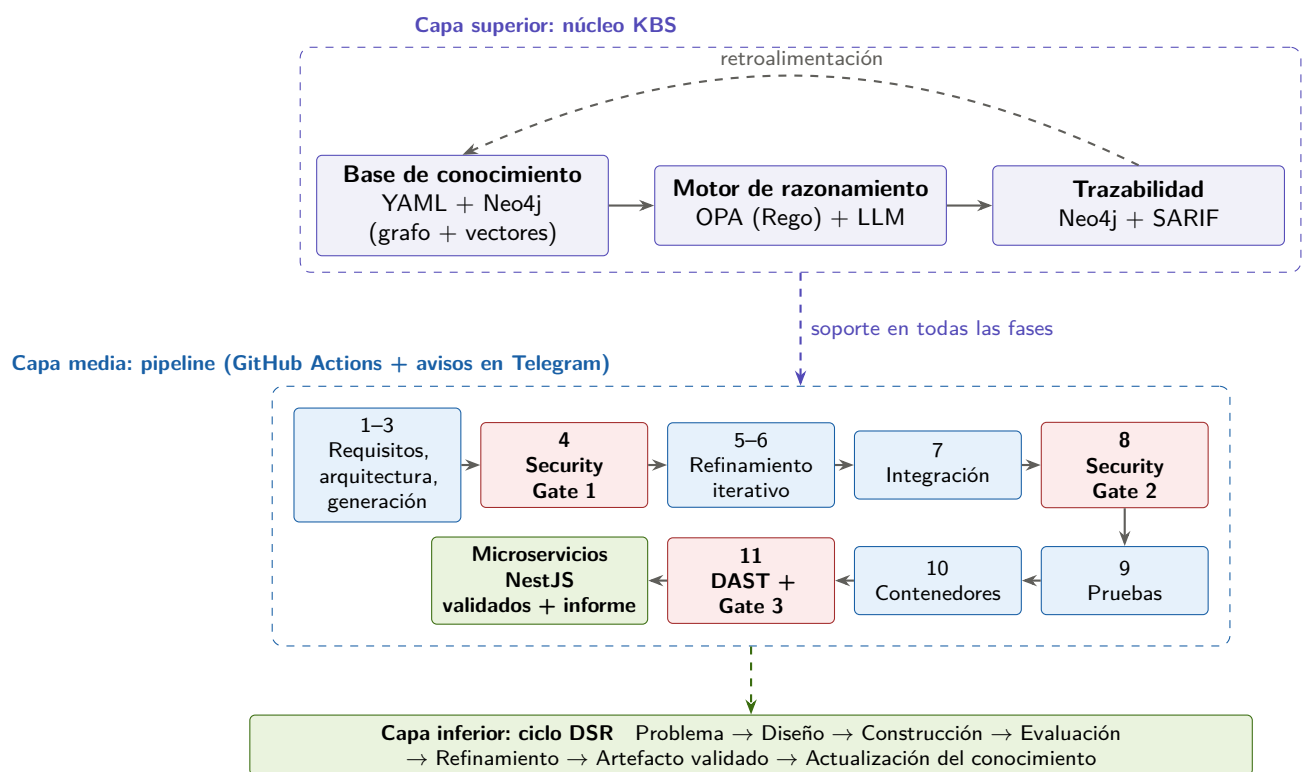


Figura 3.1: Vista general del framework y de las tecnologías de cada capa.

3.2 Vista de despliegue: dónde corre cada cosa

Una decisión crítica es dónde se ejecuta el motor. El pipeline necesita ejecutar Docker (para levantar los microservicios, escanear imágenes y lanzar el análisis dinámico), por lo que **no puede alojarse en plataformas serverless** como Vercel ni en planes gratuitos que suspenden el servicio.

Componente	Dónde se ejecuta	Motivo
Motor (orquestador, Python)	Máquina local del equipo y runners de GitHub Actions	Necesita Docker; los runners de GitHub traen Docker preinstalado y son gratuitos en repositorios públicos.

Componente	Dónde se ejecuta	Motivo
Microservicios generados (NestJS)	Docker Compose (local o en el runner)	Entorno controlado, aislado y reproducible para pruebas y DAST. Cada servicio tiene su propia base PostgreSQL.
Neo4j	Contenedor en el Docker Compose del framework	Grafo de conocimiento, trazabilidad e índice vectorial en una sola base, sin servicios externos.
Dashboard React	Archivos estáticos servidos por la API del motor	Se compila con Vite y se sirve junto al motor; no necesita servidor propio.
Bot de Telegram	API de Telegram (nube)	El motor y el workflow envían mensajes por HTTPS con el token guardado en GitHub Secrets.
API de LLM	Servicio en la nube del proveedor	Se accede por SDK con la clave en variables de entorno o GitHub Secrets.

4. Stack tecnológico completo

4.1 Resumen del stack

La tabla 4.1 resume todas las herramientas. Los capítulos siguientes justifican cada una en detalle.

Tabla 4.1: Stack tecnológico del framework.

Área	Herramienta	Función	Licencia
Núcleo de conocimiento			
Conocimiento	YAML en Git	Patrones, controles, mapeos OWASP/CWE, umbrales	—
Grafo y vectores	Neo4j Community	Grafo de conocimiento, trazabilidad e índice vectorial para RAG	GPL 3.0
Razonamiento	Open Policy Agent (Rego)	Inferencia determinista y decisión de los gates	Apache 2.0
IA generativa	SDK del proveedor elegido tras adaptador propio	Extracción, generación, corrección y explicación	Comercial (API)
Motor del framework (Python)			
Lenguaje	Python 3.12+	Lenguaje del motor	PSF
API	FastAPI + Pydantic v2	API del motor y validación estricta de datos	MIT
CLI	Typer	Ejecutar el framework desde terminal y CI	MIT
Datos	Driver oficial de Neo4j para Python	Consultas Cypher desde el motor	Apache 2.0
Gestión de dependencias	uv	Entornos y dependencias rápidos y reproducibles	MIT / Apache
Notificaciones	httpx + API de bots de Telegram	Progreso del pipeline en un chat	BSD / gratuito
Microservicios generados (TypeScript)			
Framework	NestJS	Estructura por capas, OpenAPI, transporte RabbitMQ	MIT
Validación	class-validator + class-transformer	Validación de DTO de entrada	MIT
Persistencia	Prisma + PostgreSQL	Acceso tipado a datos, una base por servicio	Apache 2.0
Documentación de API	@nestjs/swagger	Contrato OpenAPI automático	MIT
Pipeline (fases 1–11)			
Fase 1	Docling	Leer PDF, DOCX y TXT de requisitos	MIT
Fase 2	Mermaid + ADRs	Diagramas C4 y registro de decisiones	MIT
Fase 3	Copier + Jinja2	Plantilla del microservicio NestJS	MIT

Área	Herramienta	Función	Licencia
Fase 3	Compilador de TypeScript (tsc)	Verificar que el código generado compila	Apache 2.0
Fase 4	Semgrep CE / Opengrep	SAST con reglas propias para TypeScript	LGPL 2.1
Fase 4	ESLint + eslint-plugin-security	Patrones inseguros propios de JavaScript	MIT / Apache 2.0
Fase 4	Gitleaks	Detección de secretos	MIT
Fase 4	Trivy (fs , config)	SCA de dependencias npm y configuraciones	Apache 2.0
Fase 4	Hadolint	Buenas prácticas en Dockerfile	GPL 3.0
Fases 5–6	Máquina de estados en Python + ts-morph	Refinamiento iterativo con salvaguardas	– / MIT
Fase 7	Traefik	API Gateway y service discovery	MIT
Fase 7	RabbitMQ	Event bus	MPL 2.0
Fase 7	Keycloak	Identity Provider (OAuth2/OIDC)	Apache 2.0
Fase 8	Spectral + reglas OWASP API	Seguridad del contrato OpenAPI	Apache 2.0
Fase 8	Conftest (Rego)	Políticas de integración	Apache 2.0
Fase 9	Jest	Pruebas unitarias y cobertura	MIT
Fase 9	Supertest	Pruebas end-to-end de la API	MIT
Fase 9	Testcontainers (Node)	Integración con PostgreSQL y RabbitMQ reales	MIT
Fase 9	Schemathesis	Pruebas de contrato y fuzzing de API	MIT
Fase 10	Docker + Trivy image	Construcción y escaneo de imágenes	Apache 2.0
Fase 10	Syft	SBOM en CycloneDX	Apache 2.0
Fase 11	OWASP ZAP (API scan)	DAST guiado por el contrato OpenAPI	Apache 2.0
Fase 11	Nuclei	DAST por plantillas (CVEs y configuraciones)	MIT
Infraestructura, interfaz y reportes			
CI/CD	GitHub Actions	Ejecutar el pipeline, un paso por fase	Gratuito
Contenedores	Docker + Docker Compose	Entorno de ejecución y red de microservicios	Apache 2.0
Interfaz	React + Vite	Dashboard del framework	MIT
Reportes	Jinja2 + WeasyPrint	Informe de evidencia en HTML y PDF	BSD
Observabilidad	Logs JSON (structlog)	Registro estructurado del motor	MIT / Apache
Gestión del trabajo	GitHub Projects	Backlog y tablero de sprints	Gratuito

4.2 Lenguajes de programación: dónde se usa cada uno

El proyecto usa lenguajes en lugares distintos. Es importante no confundir el lenguaje del *framework* con el lenguaje del *código que el framework genera*.

Parte	Lenguaje	Justificación
Motor del framework	Python	Todo el ecosistema del núcleo es nativo de Python: SDKs de LLM, Pydantic para salidas estructuradas, Docling, driver de Neo4j, generación de PDF. Es además el estándar académico en IA y seguridad.
Microservicios generados	TypeScript (NestJS)	Requisito del proyecto. Es la familia de JavaScript con tipos: se compila a JavaScript y corre en Node.js. Los tipos permiten que el compilador rechace código erróneo del LLM antes de cualquier escaneo.
Dashboard del framework	TypeScript (React + Vite)	Coherente con el conocimiento del equipo en JS/TS. Es una herramienta del framework, no parte de su salida.
Consultas al grafo	Cypher	Lenguaje de consultas de Neo4j.
Reglas y políticas	Rego, YAML	Lenguajes declarativos de OPA y Semgrep; son la forma computacional del conocimiento.
Plantillas	Jinja2	Usado por Copier (aunque los archivos generados sean TypeScript) y por el generador de informes.
Pipeline	YAML (GitHub Actions)	Definición declarativa de los workflows de CI.

¿Por qué no generar JavaScript plano?

TypeScript cumple la indicación de usar la familia de JavaScript y añade una salvaguarda gratuita: `tsc --noEmit` detecta errores de tipos, importaciones inexistentes y firmas incorrectas en segundos. Con JavaScript plano esos errores solo aparecerían al ejecutar las pruebas o en producción.

4.2.1 Por qué NestJS y no otro framework

Alternativa	Ventaja	Por qué no se eligió
NestJS (elegido)	Organiza el código en módulos, entidades, DTO, servicios y controladores, igual que la fase 3 del diagrama; OpenAPI con <code>@nestjs/swagger</code> ; validación con <code>class-validator</code> ; inyección de dependencias; transporte nativo para RabbitMQ.	–
Express	El más conocido y simple.	Sin estructura ni validación; el LLM produciría código menos uniforme y más difícil de verificar con reglas.

Fastify / Hono	Muy rápidos y ligeros.	Pocas convenciones, lo que dificulta mantener una plantilla y reglas estables.
FastAPI (Python)	Excelente y ya usado en el motor.	No cumple la indicación de generar código de la familia JavaScript.

4.2.2 Por qué Prisma

Prisma define el modelo de datos en un archivo de esquema legible, genera un cliente totalmente tipado y gestiona las migraciones. Usa consultas parametrizadas por defecto, lo que reduce el riesgo de inyección SQL en el código generado. *TypeORM* ofrece el patrón repositorio nativo de NestJS pero con más configuración y más formas de escribir consultas inseguras; *Drizzle* es más ligero pero con menos documentación.

4.3 Interfaz del framework: React + Vite

El dashboard permite subir el documento de requisitos, seguir el progreso por fase, revisar decisiones y descargar el entregable. Se construye con **React + Vite** y se compila a archivos estáticos que sirve la API FastAPI del motor.

Criterio	React + Vite (elegido)	Next.js	Streamlit
Lenguaje	TypeScript	TypeScript	Python
Despliegue	Estáticos servidos por el motor	Servidor Node o Vercel	Junto al motor
Complejidad	Baja; sin renderizado en servidor	Media; enrutado y servidor propios	Muy baja
Progreso en tiempo real	Server-Sent Events desde FastAPI	Igual	Refresco periódico
Motivo	Equipo trabaja en JS/TS; no necesita SEO ni servidor	Funciones que el dashboard no usa	Plan de contingencia si falta tiempo

5. Núcleo de conocimiento (capa superior)

5.1 Base de conocimiento

5.1.1 Qué contiene

Tipo de conocimiento	Contenido	Formato
Patrones de microservicios	API Gateway, Database per Service, Circuit Breaker, Saga, Event-Driven, BFF; con condiciones de aplicación.	YAML → nodos Patron
Directrices Secure by Design	Mínimo privilegio, defensa en profundidad, validación de entrada, fallo seguro.	YAML → nodos Directriz
Controles de seguridad	Controles de OWASP ASVS asociados a requisitos y componentes.	YAML → nodos Control
Catálogos de vulnerabilidades	OWASP Top 10, OWASP API Security Top 10, CWE, referencias a CVE.	YAML + texto con embeddings
Reglas arquitectónicas	Restricciones de diseño (p.ej., ningún servicio accede a la base de otro).	Rego
Reglas de código	Patrones inseguros de TypeScript y convenciones propias.	Reglas Semgrep / ESLint
Políticas de aceptación	Umbral de cada gate (p.ej., cero hallazgos críticos o altos).	Rego
Dependencias permitidas	Paquetes npm aprobados para el código generado.	YAML
Decisiones validadas	Decisiones de ejecuciones anteriores con su resultado.	Nodos en Neo4j

5.1.2 Cómo se almacena: grafo de conocimiento en Neo4j

El conocimiento se escribe y versiona en Git y se carga a **Neo4j** al iniciar el motor. Cada elemento se convierte en un nodo y sus vínculos en relaciones: un patrón *mitiga* una CWE, una regla Semgrep *detecta* esa CWE, un control ASVS *se implementa con* un patrón. Los documentos de referencia (descripciones de CWE, guías OWASP) se dividen en fragmentos, se les calcula un embedding y se guardan como nodos **Fragmento** enlazados a la CWE o control que describen, con un índice vectorial nativo.

Listing 5.1: Ejemplo de patrón en la base de conocimiento (knowledge_base/patrones/api_gateway.yaml).

```
id: arq.gateway.001
patron: api_gateway
descripcion: Punto único de entrada para los clientes
aplica_cuando:
  - etiqueta: punto_unico_acceso
  - etiqueta: autenticacion_centralizada
  - numero_servicios_minimo: 2
controles_asociados: [ASVS-V4.1.1, ASVS-V13.1.3]
cwe_mitigadas: [CWE-284, CWE-306]
implementacion: traefik
```

Listing 5.2: Índice vectorial y consulta GraphRAG en Neo4j (Cypher).

```
// Índice vectorial nativo (se crea una vez)
CREATE VECTOR INDEX fragmento_embedding IF NOT EXISTS
FOR (f:Fragmento) ON (f.embedding)
OPTIONS { indexConfig: {
  `vector.dimensions`: 768,
  `vector.similarity_function`: 'cosine' } };

// GraphRAG: fragmentos similares + conocimiento conectado
CALL db.index.vector.queryNodes('fragmento_embedding', 5, $embedding)
YIELD node AS f, score
MATCH (f)-[:DESCRIBE]->(c:CWE)
OPTIONAL MATCH (p:Patron)-[:MITIGA]->(c)
OPTIONAL MATCH (ctl:Control)-[:SE_IMPLEMENTA_CON]->(p)
RETURN f.texto, c.id, collect(DISTINCT p.id) AS patrones,
       collect(DISTINCT ctl.id) AS controles, score
ORDER BY score DESC;
```

La segunda consulta muestra la ventaja de GraphRAG frente a la búsqueda vectorial sola: además del fragmento de texto, el LLM recibe los patrones y controles conectados a la vulnerabilidad, de modo que su propuesta se apoya en el conocimiento formal del framework. Las dimensiones del índice dependen del modelo de embeddings elegido y se fijan en la configuración.

5.1.3 Por qué Neo4j

Alternativa	Ventaja	Valoración
Neo4j (elegido)	El conocimiento de seguridad es un grafo por naturaleza (patrón–CWE–regla–control–requisito); recorridos de varios saltos sin joins; índice vectorial nativo que permite GraphRAG; Neo4j Browser muestra la trazabilidad de forma visual, útil en la defensa.	Elegido como base única del framework.
PostgreSQL + pgvector	SQL conocido, buenas agregaciones, búsqueda vectorial madura.	Válido, pero modelar relaciones de varios niveles requiere consultas recursivas más complejas; no aporta visualización del grafo.
Neo4j + PostgreSQL a la vez	Cada base en lo que hace mejor.	Dos bases que sincronizar y mantener; no se justifica al volumen del proyecto.
Chroma / Qdrant	Especializadas en vectores.	Solo resuelven el RAG; harían falta otra base para el grafo.
Ontología OWL/Protégé	Máxima formalidad semántica.	Curva de aprendizaje alta y difícil de integrar en CI.

5.1.4 ¿Cuál es más rápido?

A la escala de este proyecto (miles de nodos, no millones), **ambas opciones responden en milisegundos**. El tiempo total del pipeline lo dominan las llamadas al LLM (segundos por llamada), los escáneres y el DAST, por lo que la base de datos no cambia de forma apreciable la velocidad del framework. Por tipo de consulta:

Consulta	Más rápido	Motivo
Recorridos de varios saltos (cadena de trazabilidad, controles de una CWE)	Neo4j	Sigue las relaciones directamente; la ventaja crece con la profundidad.
Búsqueda vectorial para RAG	Similar	Ambas usan índices HNSW.
Agregaciones para métricas	PostgreSQL	SQL está optimizado para agrupar y contar; en Neo4j son igualmente viables a este volumen.

Decisión y evidencia

Se elige Neo4j por su ajuste al modelo de datos, no por velocidad. Para respaldar la decisión con datos, en el sprint 2 se ejecuta una prueba comparativa breve: la misma base de conocimiento cargada en Neo4j y en PostgreSQL + pgvector, midiendo la latencia media y del percentil 95 de tres consultas (cadena de trazabilidad de un requisito, controles que mitigan una CWE a tres saltos y los 5 fragmentos más similares). El resultado se documenta como anexo de la tesis.

Aclaración: PostgreSQL sigue existiendo, pero en otro lugar

Neo4j es la base del *framework*. Los *microservicios generados* siguen usando PostgreSQL (una base por servicio, patrón Database per Service), accedida con Prisma. Son dos usos distintos que no se mezclan.

5.2 Motor de razonamiento

El motor combina dos componentes con responsabilidades separadas:

- **Open Policy Agent (OPA):** razonamiento determinista. Recibe un JSON (modelo de requisitos, arquitectura propuesta, hallazgos SARIF, configuración, resultados de consultas a Neo4j) y evalúa políticas Rego que devuelven decisiones con su justificación.
- **LLM:** razonamiento generativo. Produce propuestas, código, correcciones y explicaciones en lenguaje natural, siempre validadas después por OPA y por las herramientas.

Listing 5.3: Inferencia de patrones con justificación (policies/arquitectura/patrones.rego).

```
package arquitectura.patrones

import rego.v1

recomendaciones contains rec if {
  some r in input.requisitos
  "punto_unico_acceso" in r.etiquetas
  count(input.servicios) >= 2
  rec := {
    "patron": "api_gateway",
    "regla": "arq.gateway.001",
    "justificacion": sprintf("Se aplica API Gateway porque %s exige un punto único de acceso", [r.id]),
  }
}
```

Listing 5.4: Política de aceptación del Security Gate 1 (policies/gates/gate1.rego).

```
package gates.gate1
```

```

import rego.v1

default aprobado := false

aprobado if count(bloqueantes) == 0

bloqueantes contains h if {
    some h in input.hallazgos
    h.severidad in {"critica", "alta"}
}

bloqueantes contains h if {
    some h in input.hallazgos
    h.herramienta == "gitleaks"
}

bloqueantes contains {"herramienta": "tsc", "mensaje": "el código no compila"} if {
    not input.compilacion_ok
}

```

5.2.1 Por qué OPA

Alternativa	Ventaja	Por qué no se eligió
OPA / Rego (elegido)	Estándar CNCF graduado; el mismo motor sirve para inferencia de patrones, gates y políticas de configuración (vía Conftest); decisiones explicables y testeables con <code>opa test</code> .	—
Motor de reglas en Python (experta, <code>durable_rules</code>)	Mismo lenguaje que el motor.	Proyectos con poco mantenimiento; no reutilizables para validar configuraciones.
Reglas en Cypher dentro de Neo4j	Sin componente adicional.	Mezcla datos y decisiones; Cypher no está pensado para políticas ni tiene un sistema de pruebas como <code>opa test</code> .
Solo LLM	Flexible, sin escribir reglas.	No determinista ni auditable; incompatible con el principio P1.

5.3 Trazabilidad y retroalimentación

Cada ejecución registra en Neo4j la cadena completa desde el requisito hasta la evidencia. Como la base de conocimiento está en el mismo grafo, cada decisión queda enlazada directamente a la regla que la justifica (figura 5.1).

Listing 5.5: Cadena de trazabilidad completa de un requisito (Cypher).

```

MATCH ruta = (r:Requisito {id: 'RF-01', ejecucion_id: $ejecucion})
    -[:ORIGINA]->(d:Decision)-[:IMPLEMENTA_EN]->(c:Componente)
    -[:TIENE]->(h:Hallazgo)-[:CORREGIDO_EN]->(i:Iteracion)
OPTIONAL MATCH (d)-[:BASADA_EN]->(regla:Regla)
RETURN ruta, regla.id;

```

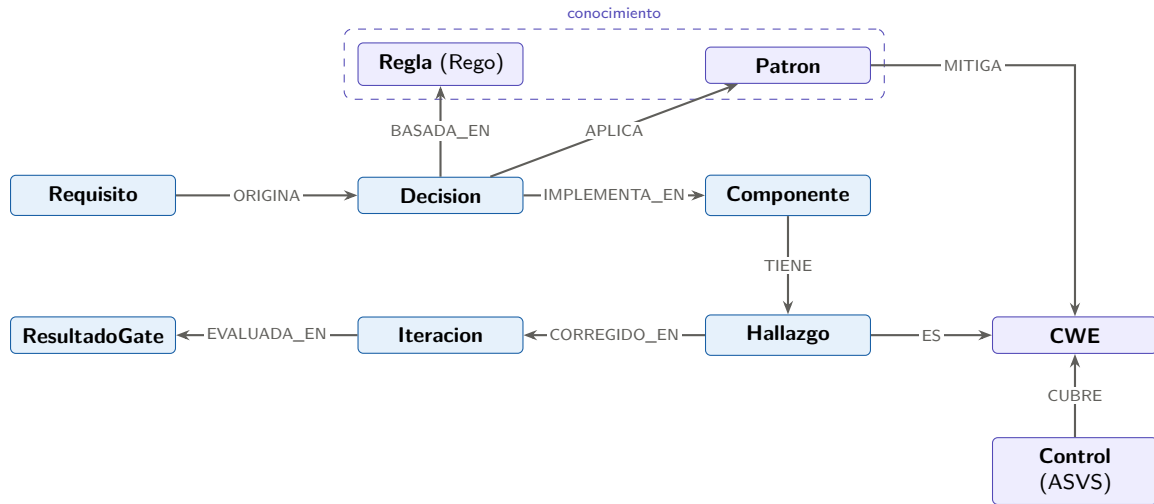


Figura 5.1: Modelo de grafo de trazabilidad (simplificado). Azul: nodos de cada ejecución; morado: nodos de la base de conocimiento. Todos los nodos de ejecución tienen `ejecucion_id` y la versión del conocimiento usada.

La **retroalimentación** cierra el ciclo: al final de cada ejecución, las correcciones que resolvieron un hallazgo sin romper pruebas se guardan como nodos **DecisionValidada**. Si una misma corrección se repite, se propone al equipo convertirla en una regla nueva de la base de conocimiento, que se revisa por pull request antes de incorporarse (aprendizaje supervisado por humanos).

5.4 SARIF como formato común

SARIF (Static Analysis Results Interchange Format) es un estándar OASIS para resultados de análisis. La mayoría de las herramientas elegidas lo generan de forma nativa (Sengrep, ESLint mediante un formateador, Gitleaks, Trivy, Hadolint, Nuclei y Spectral); para las demás (Schemathesis y, según la versión, ZAP) se escribe un conversor pequeño a partir de su salida JSON. Ventajas:

- Un solo parser en el motor para todas las herramientas.
- Los *fingerprints* identifican el mismo hallazgo entre iteraciones, lo que permite medir qué se corrigió y detectar regresiones.
- GitHub muestra los archivos SARIF en la pestaña de seguridad del repositorio sin trabajo adicional.

6. Uso de modelos de lenguaje (LLM)

6.1 Principio de uso

Los LLM se usan únicamente en tareas donde el lenguaje natural o la generación de código aportan valor, y siempre con tres condiciones:

1. **Salida estructurada:** toda respuesta se valida contra un esquema Pydantic. Si no cumple, se reintenta; nunca se procesa texto libre.
2. **Contexto desde la base de conocimiento:** el prompt incluye las reglas, controles y fragmentos OWASP/CWE relevantes recuperados por GraphRAG desde Neo4j, no el conocimiento genérico del modelo.
3. **Verificación posterior:** lo que produce el LLM pasa siempre por el compilador de TypeScript, por OPA, por las herramientas de análisis o por las pruebas. El LLM nunca tiene la última palabra.

6.2 Mapa de uso por fase

Tabla 6.1: Dónde y para qué se usa cada nivel de modelo.

Fase	Tarea del LLM	Nivel	Verificación
1	Extraer requisitos del texto, clasificar RF/RNF, identificar actores, entidades y etiquetas de seguridad	Rápido	Esquema Pydantic Requisito ; revisión opcional en el dashboard
2	Proponer bounded contexts y división en microservicios	Razonamiento	OPA valida reglas arquitectónicas e infiere patrones
2	Redactar el texto de los ADR a partir de las decisiones de OPA	Rápido	La decisión ya la tomó OPA; el LLM solo redacta
3	Completar la lógica de negocio en TypeScript dentro de la plantilla NestJS	Razonamiento	tsc , Semgrep, ESLint, pruebas, lista de dependencias permitidas
3	Generar pruebas unitarias en Jest	Razonamiento	Ejecución con Jest
5–6	Proponer corrección de un hallazgo SAST/SCA/DAST	Razonamiento	tsc , reescaneo, pruebas y control de regresiones

Fase	Tarea del LLM	Nivel	Verificación
5-6	Explicar el hallazgo y la corrección para el informe	Rápido	Contenido informativo, no decisorio
Transversal	Embeddings de OWASP, CWE y guías para GraphRAG	Embeddings	Evaluación de recuperación con consultas de prueba
11	Resumen ejecutivo del informe final	Rápido	Datos numéricos provienen de Neo4j, no del LLM

Dónde NO se usa el LLM

Aprobar o rechazar un Security Gate, asignar severidades, calcular métricas, decidir si una prueba pasó y seleccionar el patrón arquitectónico final. Estas decisiones son deterministas y se toman con OPA, las herramientas y consultas a Neo4j.

6.3 Selección del modelo

El proveedor se elige en el sprint 1 con una prueba corta: los mismos cinco requisitos se procesan con dos o tres candidatos y se compara la calidad del JSON extraído y del código TypeScript generado. Los criterios son:

1. **Salida estructurada** con esquemas JSON, imprescindible para Pydantic.
2. **Calidad generando TypeScript y NestJS**, medida por cuántos archivos compilan sin cambios.
3. **Ventana de contexto amplia**, para enviar archivos completos y el contexto de GraphRAG.
4. **Coste y plan gratuito** suficientes para desarrollo y experimentos.
5. **Latencia**, que influye directamente en el tiempo total del pipeline.

Candidatos de referencia: Gemini (plan gratuito amplio y salida estructurada), Claude, GPT y un modelo abierto local ejecutado con Ollama como opción sin coste.

- **Dos niveles de modelo:** uno rápido y económico para extracción, clasificación y redacción, y uno de razonamiento para arquitectura, generación y corrección de código. Las versiones concretas cambian con frecuencia; al elegir el proveedor se fija la versión vigente y se registra en la configuración, para que los experimentos sean reproducibles.
- **Embeddings:** el modelo de embeddings del mismo proveedor, o un modelo abierto multilingüe ejecutado localmente (por ejemplo, de la familia multilingual-e5) si se quiere evitar costes y dependencia externa. Su dimensión define la del índice vectorial de Neo4j.
- **Temperatura baja** (0 a 0,2) en extracción y corrección para reducir la variabilidad; se documenta en cada experimento.

6.4 Adaptador de proveedor

El motor no llama al SDK directamente desde las fases, sino a través de una interfaz propia. Esto permite cambiar de proveedor con un parámetro de configuración y, sobre todo, **comparar modelos en la evaluación DSR**, lo que enriquece los resultados de la tesis.

Listing 6.1: Interfaz del adaptador de LLM (orchestrator/llm/base.py).

```

from typing import Protocol, TypeVar
from pydantic import BaseModel

T = TypeVar("T", bound=BaseModel)

class ProveedorLLM(Protocol):
    def generar_estructurado(
        self, prompt: str, esquema: type[T], nivel: str = "rapido"
    ) -> T:
        """Devuelve una instancia validada del esquema o lanza error."""
        ...

```

Alternativa	Ventaja	Por qué no se eligió
SDK oficial + adaptador propio (elegido)	Control total, fácil de depurar, sin dependencias pesadas.	–
LangChain / LangGraph	Muchos componentes prefabricados, grafos de agentes.	Capa de abstracción grande que dificulta depurar; la lógica de este proyecto es un flujo conocido que no necesita agentes autónomos.
LiteLLM	Interfaz unificada para muchos proveedores.	Opción válida si se quieren comparar muchos modelos; se puede usar dentro del adaptador.

6.5 Riesgos específicos del uso de LLM

Riesgo	Mitigación
Prompt injection desde el documento de requisitos	El documento se trata como dato, delimitado en el prompt; la salida se valida con Pydantic; el LLM no tiene acceso a herramientas ni a ejecución.
Paquetes npm alucinados (<i>slopsquatting</i>)	Riesgo especialmente alto en el ecosistema npm. Toda dependencia generada se valida contra la lista permitida, se instala con <code>npm ci --ignore-scripts</code> y pasa por SCA.
Código que no compila	<code>tsc --noEmit</code> rechaza el código antes de cualquier escaneo.
Corrección que elimina funcionalidad	Salvaguardas del motor de refinamiento (sección 7.5).
Variabilidad entre ejecuciones	Temperatura baja, versión de modelo fijada, varias repeticiones por experimento y registro de prompts y respuestas.
Fuga de datos sensibles	No se envían secretos ni datos reales; las claves se gestionan en variables de entorno y GitHub Secrets.
Coste y límites de uso	Caché de respuestas por hash del prompt durante el desarrollo; modelo rápido por defecto.

7. Pipeline de construcción y validación (capa media)

Cada fase se describe con el mismo formato: objetivo, herramientas, entradas, salidas y justificación frente a alternativas. Cada fase notifica su inicio y su resultado en Telegram (capítulo 8).

7.1 Fase 1: Adquisición de requisitos

Objetivo: convertir documentos en lenguaje natural en un modelo estructurado de requisitos.

Paso	Herramienta y uso
Captura	Carga en el dashboard, por CLI (<code>framework run requisitos.pdf</code>) o subiendo el archivo a <code>requirements/</code> en el repositorio.
Extracción	Docling convierte PDF y DOCX a texto estructurado conservando títulos, listas y tablas.
Normalización	Limpieza en Python (espacios, numeración, división en unidades de requisito).
Análisis semántico y clasificación	LLM rápido con esquema Pydantic: tipo (RF/RNF), actor, acción, entidades, servicio sugerido y etiquetas de seguridad.
Registro	Cada requisito se guarda como nodo Requisito en Neo4j.
Validación	Pydantic rechaza salidas incompletas; el dashboard permite revisar y corregir antes de continuar.

Entrada: PDF, DOCX, TXT o historias de usuario. **Salida:** `modelo_requisitos.json` y nodos en Neo4j.

Listing 7.1: Ejemplo de requisito estructurado.

```
{
  "id": "RF-01",
  "tipo": "funcional",
  "actor": "cliente",
  "accion": "registrarse con correo",
  "entidades": ["Usuario"],
  "servicio_sugerido": "usuarios",
  "etiquetas": ["autenticacion", "validacion_entrada", "hash_password"]
}
```

Por qué Docling: proyecto abierto de IBM que maneja bien la estructura de documentos (tablas, títulos, listas) y exporta a Markdown o JSON. Frente a *PyPDF* solo extrae texto plano; frente a *Unstructured* es más ligero de instalar para este caso de uso.

7.2 Fase 2: Análisis y propuesta arquitectónica

Objetivo: decidir qué microservicios crear, qué patrones aplicar y por qué.

1. El LLM de razonamiento propone bounded contexts y la división en servicios a partir del modelo de requisitos.

2. OPA evalúa la propuesta contra las reglas arquitectónicas (p.ej., cada servicio con su propia base de datos, sin dependencias cíclicas) e infiere los patrones y controles aplicables, con su justificación.
3. Si OPA rechaza la propuesta, el motivo se devuelve al LLM para una nueva propuesta (máximo tres intentos).
4. Las decisiones se guardan como nodos **Decision** enlazados a sus requisitos y reglas.
5. Se generan los ADR (Architecture Decision Records) en Markdown y el diagrama C4 en **Mermaid**.

Salida: `arquitectura.json`, `docs/adr/*.md`, `docs/arquitectura.md`.

Por qué ADR + Mermaid: los ADR son el formato estándar de la industria para documentar decisiones con contexto y consecuencias, y encajan con la trazabilidad. Mermaid se escribe como texto, se versiona en Git y GitHub lo renderiza sin herramientas adicionales. Structurizr es más completo para C4 pero añade otra herramienta y otro lenguaje.

7.3 Fase 3: Generación de la estructura interna

Objetivo: producir el código TypeScript de cada microservicio NestJS.

- **Copier** genera el esqueleto determinista desde la plantilla NestJS: módulo, entidades (esquema Prisma), DTO con `class-validator`, servicio, controlador con decoradores de Swagger, configuración, Dockerfile y pruebas Jest base.
- La plantilla ya incluye los controles transversales: guard de autenticación JWT validado contra Keycloak, `ValidationPipe` global con lista blanca de propiedades, cabeceras de seguridad (Helmet), filtro de excepciones que no expone detalles internos, logging estructurado y configuración por variables de entorno.
- El LLM solo completa la lógica de negocio en puntos marcados de la plantilla.
- Al terminar, `tsc --noEmit` verifica que todo compila; si no, el error vuelve al LLM antes de pasar al Gate 1.

Listing 7.2: Estructura de un microservicio generado.

```
services/usuarios/
|-- src/
|   |-- usuarios.module.ts
|   |-- usuarios.controller.ts      # endpoints + decoradores Swagger
|   |-- usuarios.service.ts        # lógica de negocio (LLM)
|   |-- dto/crear-usuario.dto.ts    # validación con class-validator
|   |-- auth/jwt.guard.ts           # validación de tokens de Keycloak
|   |-- main.ts                    # Helmet, ValidationPipe, Swagger
|-- prisma/schema.prisma           # entidades y migraciones
|-- test/                          # Jest y Supertest
|-- Dockerfile
|-- package.json
`-- package-lock.json
```

Alternativa	Ventaja	Por qué no se eligió
-------------	---------	----------------------

Copier (elegido)	Plantillas Jinja2 que pueden generar cualquier tipo de archivo, preguntas tipadas y capacidad de <i>actualizar</i> proyectos ya generados cuando cambia la plantilla. Se invoca desde el motor en Python.	–
Nest CLI (nest generate)	Generadores oficiales de NestJS.	Producen esqueletos genéricos sin los controles de seguridad del proyecto; habría que modificarlos después.
Plop / Hygen	Generadores populares en el ecosistema JS.	Requieren invocar Node desde el motor; Copier se integra directamente en Python.
Generación 100 % por LLM	Máxima flexibilidad.	Resultados inconsistentes y difíciles de validar; más vulnerabilidades.

Decisión clave

La combinación *plantilla determinista + LLM acotado + compilador de TypeScript* reduce drásticamente la superficie de error y permite medir en la evaluación cuánto aporta cada parte.

7.4 Fase 4: Security Gate 1, análisis estático

Herramienta	Qué analiza	Ventaja frente a alternativas
tsc --noEmit	Compilación y tipos	Primer filtro, el más barato; código que no compila no se escanea.
Semgrep CE u Opengrep	SAST: inyecciones, criptografía débil, SSRF, deserialización insegura y reglas propias para NestJS	Muy rápido; reglas en YAML legibles que forman parte de la base de conocimiento. SonarQube requiere servidor y base de datos propios.
ESLint + eslint-plugin-security	Patrones inseguros propios de JavaScript (expresiones regulares vulnerables, <code>eval</code> , rutas dinámicas)	Complemento ligero, conocido por cualquier equipo JS.
Gitleaks	Secretos (claves, tokens, contraseñas)	Binario único, milisegundos por escaneo, reglas configurables. Más ligero que TruffleHog para este uso.
Trivy fs	SCA: vulnerabilidades en dependencias npm a partir de <code>package-lock.json</code>	Misma herramienta que en la fase 10. <code>npm audit</code> es equivalente pero no genera SARIF ni cubre configuraciones.

Trivy config	Configuraciones inseguras en Dockerfile y Compose	Misma herramienta, sin instalar Checkov por separado.
Hadolint	Buenas prácticas en Dockerfile	Estándar de facto, salida SARIF, muy rápido.

Nota de licencia de Semgrep

En diciembre de 2024 Semgrep cambió la licencia de sus reglas oficiales y trasladó funciones a su edición comercial; la comunidad respondió con el fork **Opengrep**, totalmente abierto. Semgrep CE sigue siendo válido para uso académico. En ambos casos se recomienda escribir y usar reglas propias del proyecto para TypeScript y NestJS, que además forman parte de la base de conocimiento.

Decisión: todos los resultados se convierten a SARIF, se envían a OPA con la política `gates.gate1` y se registran en Neo4j. Si hay bloqueantes, el flujo pasa a las fases 5–6.

7.5 Fases 5 y 6: Refinamiento iterativo guiado por conocimiento

Objetivo: corregir los hallazgos de cualquier gate sin romper la funcionalidad, dejando registro de cada paso.

El motor de refinamiento es una **máquina de estados explícita en Python**. Recibe los hallazgos, la arquitectura, el código y el contexto de GraphRAG, y sigue los pasos del diagrama del proyecto: identificar, explicar, priorizar, seleccionar control y refinar.

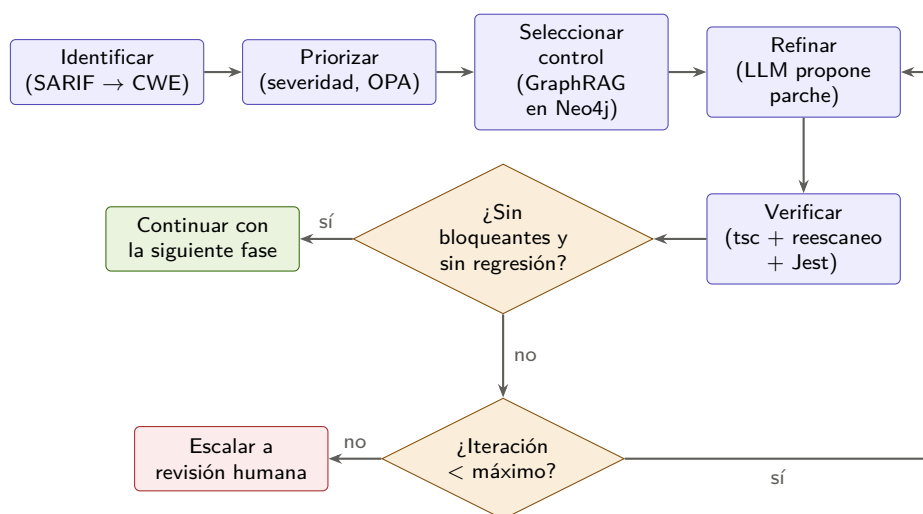


Figura 7.1: Máquina de estados del motor de refinamiento.

7.5.1 Salvaguardas obligatorias

1. **Compilación:** el parche debe pasar `tsc --noEmit`; si no compila, se rechaza sin escanear.
2. **Máximo de iteraciones** configurable (por defecto 5). Si no converge, se escala a revisión humana y queda registrado.
3. **Pruebas tras cada cambio:** si una prueba Jest que antes pasaba ahora falla, el parche se rechaza.

4. **Control de regresión por fingerprint SARIF:** un parche que introduce hallazgos nuevos se rechaza.
5. **Integridad funcional:** se rechazan parches que eliminan clases, métodos, endpoints o pruebas. Se verifica con **ts-morph**, que analiza el árbol sintáctico de TypeScript, invocado desde el motor mediante un script de Node.
6. **Alcance acotado:** el parche solo puede modificar los archivos implicados en el hallazgo.
7. **Dependencias:** solo se aceptan paquetes y versiones de la lista permitida.
8. **Registro completo:** cada iteración guarda el diff, el prompt, la respuesta y el resultado como nodo **Iteracion** en Neo4j, y se notifica en Telegram.

Qué puede modificar: código, configuración, dependencias (solo hacia versiones de la lista permitida), contratos de API y, en casos justificados, decisiones arquitectónicas (lo que devuelve el flujo a la fase 2).

Por qué una máquina de estados propia y no LangGraph: el flujo es conocido y finito; una implementación de menos de 300 líneas en Python es más fácil de depurar, probar y explicar en la tesis que un grafo de agentes con una librería externa.

7.6 Fase 7: Integración de microservicios

Componente	Herramienta	Justificación y alternativas
API Gateway	Traefik	Se configura con etiquetas en Docker Compose, detecta servicios automáticamente, incluye TLS, rate limiting y middleware de autenticación. Kong y NGINX requieren más configuración manual.
Service Discovery	Traefik (proveedor Docker)	El descubrimiento por etiquetas cubre la necesidad sin instalar Consul o Eureka.
Load Balancer	Traefik	Balanceo integrado entre réplicas de un servicio.
Event Bus	RabbitMQ	NestJS tiene un transporte nativo para RabbitMQ en @nestjs/microservices , por lo que la plantilla lo integra con poco código. Interfaz web útil para demostraciones. NATS es más ligero; Kafka es excesivo para este alcance.
Identity Provider	Keycloak (modo desarrollo)	Estándar abierto de OAuth2/OIDC con consola de administración; el realm se genera como archivo JSON. Los servicios validan los tokens con passport-jwt . Auth0 es comercial.
Bases de datos	PostgreSQL por servicio	Patrón Database per Service; cada servicio solo accede a la suya.
Observabilidad	Logs JSON + cabecera de correlación	Suficiente para trazar peticiones entre servicios. OpenTelemetry con Grafana queda como mejora opcional.
Service Mesh	No se implementa	Fuera de alcance; se justifica como trabajo futuro (mTLS entre servicios).

Listing 7.3: Fragmento de docker-compose.yml generado.

```
services:
```

```

gateway:
  image: traefik:v3.1
  command:
    - --providers.docker=true
    - --providers.docker.exposedbydefault=false
    - --entrypoints.web.address=:80
  ports: ["80:80"]
  volumes: ["/var/run/docker.sock:/var/run/docker.sock:ro"]

usuarios:
  build: ./services/usuarios
  read_only: true
  user: "10001"
  networks: [frontal, usuarios_db]
  labels:
    - traefik.enable=true
    - traefik.http.routers.usuarios.rule=PathPrefix(`/api/usuarios`)
    - traefik.http.middlewares.ratelimit.ratelimit.average=50
    - traefik.http.routers.usuarios.middlewares=ratelimit@docker
  env_file: ./services/usuarios/.env

usuarios-db:
  image: postgres:16-alpine
  networks: [usuarios_db] # solo accesible por su servicio

```

7.7 Fase 8: Security Gate 2, seguridad de integración

Herramienta	Qué valida	Ventaja
Spectral con reglas OWASP API Security	Contratos OpenAPI generados por <code>@nestjs/swagger</code> : autenticación declarada en cada endpoint, sin HTTP plano, límites en cadenas y arreglos, respuestas de error definidas.	Valida la seguridad <i>antes</i> de levantar los servicios; las reglas son YAML.
Conftest (Rego)	Docker Compose y configuración de Traefik: TLS, middleware de autenticación, contenedores sin privilegios, sistema de archivos de solo lectura, secretos fuera de variables en claro, redes separadas por base de datos.	Reutiliza el mismo lenguaje Rego y la misma base de conocimiento que OPA.
OPA (<code>gates.gate2</code>)	Decisión final del gate a partir de ambos resultados.	Decisión explicable y registrada.

7.8 Fase 9: Pruebas automatizadas

Tipo de prueba	Herramienta	Uso y ventaja
----------------	-------------	---------------

Unitarias	Jest	Framework de pruebas por defecto de NestJS; la plantilla trae pruebas base y el LLM genera pruebas adicionales. Mide cobertura con <code>--coverage</code> . Vitest es más rápido, pero Jest viene configurado en NestJS.
Integración	Testcontainers (Node)	Levanta PostgreSQL y RabbitMQ reales dentro de la prueba; evita falsos positivos de los mocks.
Funcionales / end-to-end	Supertest	Recorren flujos completos a través del gateway (registro, login, compra); estándar en NestJS.
Contrato y fuzzing	Schemathesis	Lee el OpenAPI y genera cientos de casos, incluidos datos inválidos, verificando que las respuestas cumplan el contrato y que no haya errores 500. Funciona con cualquier lenguaje porque solo necesita el contrato. Pact es más completo para contratos consumidor-proveedor pero requiere mantener contratos manuales.
Regresión	Suite completa de Jest	Se ejecuta en cada iteración y al final.
Seguridad	Jest + Supertest con casos derivados de la base de conocimiento	Pruebas de autorización (acceso a recursos ajenos, tokens inválidos o expirados).

Criterio de paso: todas las pruebas en verde, cobertura mínima configurable (por ejemplo, 70 %) y cero errores 500 en Schemathesis, evaluado por OPA.

7.9 Fase 10: Contenerización

- **Docker:** imágenes multietapa. La etapa de construcción usa `node:22-slim`, instala con `npm ci --ignore-scripts` y compila TypeScript; la imagen final solo contiene el JavaScript compilado y las dependencias de producción, sobre una base mínima (`node:22-slim` o `distroless` de Node), con usuario no root.
- **Trivy image:** escaneo de CVEs del sistema operativo y de las librerías de la imagen. Frente a Gype es equivalente en calidad, pero Trivy ya se usa en la fase 4, lo que reduce herramientas.
- **Syft:** genera el SBOM en CycloneDX, que se adjunta al informe como inventario verificable de componentes.
- Variables de entorno y secretos: archivos `.env` excluidos de Git y secretos de Docker Compose; en CI, GitHub Secrets.

7.10 Fase 11: DAST y Security Gate 3

Los servicios se despliegan con Docker Compose en un entorno controlado (red aislada del runner) y se atacan de forma automática.

Herramienta	Qué aporta	Por qué ambas
-------------	------------	---------------

OWASP ZAP	Importa el OpenAPI y ejecuta un escaneo activo contra cada endpoint: inyección, fallos de autenticación, cabeceras, exposición de datos.	Es el DAST de referencia de OWASP y el único de los dos que prueba activamente la API a partir del contrato. Se limita a 5–10 minutos por servicio para evitar timeouts en CI.
Nuclei	Plantillas YAML para CVEs conocidas, configuraciones erróneas y paneles expuestos.	Muy rápido y ligero; complementa a ZAP con detección basada en firmas. No sustituye a ZAP porque no rastrea ni ataca la lógica de la API.
Pruebas autenticadas	Token de Keycloak inyectado en ZAP y Schemathesis	Permite probar endpoints protegidos, no solo los públicos.

Gate 3: OPA evalúa los resultados DAST junto con los de Trivy image y la fase 9. Si se aprueba, se genera el entregable final; si no, el hallazgo vuelve al motor de refinamiento.

8. Infraestructura, CI/CD y notificaciones

8.1 GitHub Actions como orquestador del pipeline

GitHub Actions ejecuta el framework completo en cada cambio de requisitos o de conocimiento. Ventajas frente a las alternativas:

- Frente a **Jenkins**: no requiere servidor propio ni mantenimiento.
- Frente a **GitLab CI**: equivalente en funciones; se elige GitHub por la integración nativa con SARIF en la pestaña de seguridad y por ser gratuito en repositorios públicos.
- Los runners traen Docker, Node.js y Python preinstalados, requisitos del pipeline.
- Con la herramienta **act** se pueden ejecutar los workflows localmente para depurar.

8.2 Un paso por fase

El workflow se divide en un paso por fase (`framework run --fase N`). Así la interfaz de GitHub muestra cada fase por separado con su duración y su estado, y cada fase puede notificar su resultado en Telegram. Si todo el framework corriera en un único paso, GitHub y Telegram solo mostrarían “ejecutando” y “terminado”.

El estado entre fases (modelo de requisitos, arquitectura, código generado, hallazgos) se conserva en el directorio de trabajo del job y en Neo4j, de modo que cada paso continúa donde terminó el anterior.

Listing 8.1: Workflow principal (.github/workflows/framework.yml), simplificado.

```
name: framework-desarrollo-seguro
on:
  push:
    paths: ["requirements/**", "knowledge_base/**", "policies/**"]
  workflow_dispatch:

env:
  TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
  TELEGRAM_CHAT_ID: ${ secrets.TELEGRAM_CHAT_ID }
  LLM_API_KEY: ${ secrets.LLM_API_KEY }
  NEO4J_URI: bolt://localhost:7687
  NEO4J_PASSWORD: ${ secrets.NEO4J_PASSWORD }

jobs:
  pipeline:
    runs-on: ubuntu-latest
    timeout-minutes: 90
    services:
      neo4j:
        image: neo4j:5-community
        env: { NEO4J_AUTH: neo4j/${ secrets.NEO4J_PASSWORD } }
        ports: ["7687:7687"]
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - name: Instalar herramientas de seguridad
        run: ./scripts/instalar_herramientas.sh
      - name: Probar políticas OPA
        run: opa test policies/ -v
```

```

- name: Iniciar notificación
  run: uv run framework notificar --inicio
- name: "Fase 1: Requisitos"
  run: uv run framework run --fase 1 --entrada requirements/
- name: "Fase 2: Arquitectura"
  run: uv run framework run --fase 2
- name: "Fase 3: Generación"
  run: uv run framework run --fase 3
- name: "Fases 4-6: Gate 1 y refinamiento"
  run: uv run framework run --fase 4
# ... fases 7 a 11 con el mismo formato ...
- name: Publicar SARIF
  if: always()
  uses: github/codeql-action/upload-sarif@v3
  with: { sarif_file: salida/security/sarif/ }
- uses: actions/upload-artifact@v4
  with: { name: entregable, path: salida/ }
- name: Aviso de fallo
  if: failure()
  run: uv run framework notificar --fallo "${{ github.server_url }}/${{ github.repository
  }}/actions/runs/${{ github.run_id }}"

```

8.3 Notificaciones en Telegram

8.3.1 Diseño

Las notificaciones se emiten desde un **módulo notificador dentro del motor**, no solo desde el workflow. Así funcionan igual al ejecutar el framework en local y en CI, y pueden informar eventos internos de cada fase (por ejemplo, cada iteración de refinamiento), que el workflow no ve.

Evento	Mensaje
Inicio de ejecución	Proyecto, documento de requisitos, enlace a la ejecución en GitHub.
Fase iniciada / terminada	Número y nombre de la fase, duración, resultado resumido (p.ej., “4 microservicios generados”).
Resultado de gate	Aprobado o rechazado, número de bloqueantes por severidad.
Iteración de refinamiento	Número de iteración y hallazgos pendientes.
Escalado a revisión humana	Aviso destacado con enlace.
Fin	Resultado final, métricas clave y enlace al entregable.
Fallo inesperado	Paso donde falló y enlace a los registros (desde el paso <code>if: failure()</code>).

8.3.2 Buenas prácticas aplicadas

- **Un mensaje de progreso que se edita** con `editMessageText`, en lugar de un mensaje por evento. Telegram limita la cantidad de mensajes por chat y por grupo, y un pipeline con varias iteraciones superaría esos límites. Se envían mensajes nuevos solo para el inicio, el resultado final, los escalados y los fallos.
- **Sin datos sensibles:** nunca se envían fragmentos de código, rutas de archivos vulnerables ni detalles explotables; solo resúmenes y el enlace a GitHub, donde el acceso está controlado.

- **Credenciales en GitHub Secrets:** TELEGRAM_BOT_TOKEN y TELEGRAM_CHAT_ID. El identificador del mensaje de progreso se guarda en \$GITHUB_ENV para que los pasos siguientes lo editen.
- **Tolerancia a fallos:** si Telegram no responde, el pipeline continúa; la notificación nunca bloquea una fase.

Listing 8.2: Notificador (orchestrator/notify/telegram.py), simplificado.

```
import os
import httpx

API = f"https://api.telegram.org/bot{os.environ['TELEGRAM_BOT_TOKEN']}"
CHAT = os.environ["TELEGRAM_CHAT_ID"]

def enviar(texto: str) -> int | None:
    try:
        r = httpx.post(f"{API}/sendMessage",
                       json={"chat_id": CHAT, "text": texto}, timeout=10)
        return r.json()["result"]["message_id"]
    except Exception:
        return None # la notificación nunca detiene el pipeline

def actualizar(message_id: int, texto: str) -> None:
    try:
        httpx.post(f"{API}/editMessageText",
                  json={"chat_id": CHAT, "message_id": message_id,
                      "text": texto}, timeout=10)
    except Exception:
        pass
```

Por qué la API de bots directamente: es una llamada HTTP simple, sin dependencias adicionales. Acciones de GitHub como `appleboy/telegram-action` sirven para avisos sueltos del workflow, pero no permiten editar un mensaje de progreso ni notificar eventos internos del motor.

8.4 Entornos

Entorno	Uso
Local	Desarrollo del framework: Docker Desktop o Docker Engine, uv, Node.js 22, herramientas instaladas por script. Recomendado 16 GB de RAM (Neo4j, Keycloak, RabbitMQ y los microservicios a la vez).
CI (GitHub Actions)	Ejecuciones reproducibles para la evaluación y demostración del DevSecOps continuo.
Entorno controlado de pruebas	Red Docker aislada donde se despliegan los microservicios generados para las fases 9 y 11.

9. Entradas y salidas del framework

9.1 Formas de entrada

1. **Dashboard (React):** se arrastra el documento, se eligen opciones (nivel de seguridad, iteraciones máximas) y se ejecuta. Ideal para demostraciones.
2. **CLI:** `framework run requisitos.pdf --nivel alto`. Ideal para experimentos por lotes.
3. **Repositorio:** subir el documento a `requirements/` dispara el workflow y las notificaciones de Telegram. Demuestra el DevSecOps continuo.

9.2 Salida: el entregable

El framework entrega únicamente backend: microservicios NestJS y su infraestructura. No genera frontend.

Listing 9.1: Estructura del entregable generado.

```
salida_proyecto/
|-- services/
|   |-- usuarios/          # NestJS: módulo, entidades Prisma, DTO, servicio, controlador, tests
|   |-- catalogo/
|   |-- pedidos/
|   `-- pagos/
|-- gateway/              # configuración de Traefik
|-- keycloak/              # realm exportado
|-- docker-compose.yml    # levanta todo el sistema con un comando
|-- openapi/              # contrato de cada microservicio
|-- docs/
|   |-- adr/               # decisiones arquitectónicas justificadas
|   `-- arquitectura.md   # diagrama C4 en Mermaid
|-- security/
|   |-- sarif/             # resultados de todas las herramientas
|   `-- sbom.json         # inventario de componentes (CycloneDX)
|-- metricas.json
`-- informe_evidencia.pdf
```

9.3 Informe de evidencia y trazabilidad

Generado con **Jinja2** (plantilla HTML) y **WeasyPrint** (conversión a PDF) a partir de consultas Cypher a Neo4j. Contiene:

- **Matriz de trazabilidad:** requisito → decisión → microservicio → archivos → pruebas → controles.
- **Decisiones arquitectónicas** con la regla de la base de conocimiento que las justifica.
- **Historial de refinamiento:** hallazgos, iteraciones y correcciones aplicadas.
- **Métricas de seguridad, calidad, arquitectura, eficiencia y efectividad** (capítulo 11).
- **Veredicto de cada Security Gate** con los bloqueantes resueltos.
- **Resumen ejecutivo** redactado por el LLM a partir de datos de Neo4j (los números nunca los genera el LLM).

10. Seguridad del propio framework

Un framework de seguridad debe aplicarse a sí mismo sus principios.

Riesgo	Control
Código generado malicioso o defectuoso	Solo se ejecuta dentro de contenedores con red aislada, usuario no root y límites de recursos.
Scripts de instalación de npm	<code>npm ci --ignore-scripts</code> en todo el pipeline; solo paquetes de la lista permitida.
Secretos del framework	Claves en GitHub Secrets y <code>.env</code> fuera de Git; Gitleaks también escanea el repositorio del framework.
Dependencias del framework	Trivy y Dependabot sobre el propio repositorio (Python y npm del dashboard).
Manipulación del conocimiento	Cambios en <code>knowledge_base/</code> y <code>policies/</code> solo por pull request revisado por el otro integrante; <code>opa test</code> obligatorio en CI.
Fuga de información por Telegram	Solo resúmenes y enlaces; el token del bot en GitHub Secrets; chat privado del equipo.
Acceso a Neo4j	Contraseña en secretos, puerto expuesto solo en local, sin acceso público.
Prompt injection	Ver capítulo de LLM.
Socket de Docker expuesto a Traefik	Montado en solo lectura; en entornos compartidos, uso de un proxy de socket.

11. Evaluación del artefacto (ciclo DSR)

11.1 Relación entre el ciclo DSR y el plan

Etapa DSR	Qué se hace en este proyecto
Problema / necesidad	Revisiones de seguridad manuales, tardías y aisladas en microservicios; falta de trazabilidad entre requisitos y controles.
Diseño del artefacto	Arquitectura de tres capas, principios P1–P5 y este plan (sprint 0).
Construcción	Implementación por sprints 1 a 7 (capítulo 13).
Evaluación	Experimentos con conjuntos de requisitos y comparación con líneas base (sprint 8).
Refinamiento	Ajuste de reglas, plantillas y prompts según los resultados (sprint 8).
Reevaluación	Repetición de los experimentos con el conocimiento actualizado (sprint 8).
Artefacto validado	Framework consolidado y documentado.
Actualización del conocimiento	Nuevas reglas y lecciones incorporadas a la base de conocimiento.

11.2 Métricas

Dimensión	Métrica	Fuente
Seguridad	Número de vulnerabilidades por severidad; riesgo residual; porcentaje reducido entre el primer gate y el final	SARIF + Neo4j
Calidad	Cobertura de pruebas; porcentaje de requisitos cubiertos; errores funcionales; porcentaje de código generado que compila al primer intento	Jest, Schemathesis, tsc , matriz de trazabilidad
Arquitectura	Porcentaje de requisitos trazados; cumplimiento de reglas arquitectónicas; acoplamiento entre servicios	OPA, grafo en Neo4j
Eficiencia	Iteraciones de refinamiento; tiempo total y tiempo hasta cada gate; tokens consumidos	Registros del motor
Efectividad	Vulnerabilidades iniciales frente a finales; porcentaje de controles de seguridad satisfechos	Neo4j

11.3 Diseño experimental

11.3.1 Conjuntos de requisitos

Se preparan tres documentos de requisitos de dominios distintos (tienda en línea, gestión académica, citas médicas), con complejidad creciente. Cada uno se ejecuta tres veces por configuración para medir la variabilidad.

11.3.2 Líneas base de comparación

Configuración	Descripción
B0: solo LLM	El LLM genera los microservicios NestJS desde los requisitos, sin plantillas, sin base de conocimiento ni gates.
B1: LLM + gates	Igual que B0 pero con los Security Gates y refinamiento, sin base de conocimiento ni GraphRAG.
F: framework completo	Plantillas, base de conocimiento en Neo4j, OPA, GraphRAG, gates y refinamiento.

La diferencia entre B0, B1 y F permite atribuir la mejora a cada componente, que es el argumento central de la tesis. Opcionalmente se repite F con distintos modelos gracias al adaptador de proveedor.

11.3.3 Calibración de las herramientas dinámicas

Para demostrar que el DAST detecta lo que debe, se ejecuta la fase 11 contra aplicaciones deliberadamente vulnerables con vulnerabilidades conocidas:

- **VAmPI**: API REST vulnerable con vulnerabilidades del OWASP API Top 10.
- **OWASP crAPI**: aplicación de microservicios vulnerable, muy cercana a la arquitectura del proyecto.

Se mide la tasa de detección frente a las vulnerabilidades documentadas de cada aplicación.

11.4 Dashboard de métricas

El dashboard en React muestra por ejecución: progreso por fase en tiempo real, decisiones tomadas con su justificación, hallazgos por gate e iteración, comparación entre configuraciones B0, B1 y F, y descarga del entregable. Para explorar la trazabilidad como grafo se usa Neo4j Browser.

12. Estructura del repositorio del framework

```
framework/
|-- knowledge_base/          # YAML: patrones, controles, mapeos CWE/OWASP, umbrales
|   |-- patrones/
|   |-- controles/
|   |-- vulnerabilidades/
|   `-- dependencias_permitidas.yaml # paquetes npm aprobados
|-- graph/                  # scripts Cypher: restricciones, índices, carga
|-- policies/              # Rego
|   |-- arquitectura/      # inferencia de patrones y reglas de diseño
|   |-- gates/             # gate1.rego, gate2.rego, gate3.rego
|   `-- integracion/       # políticas para Conftest
|-- semgrep_rules/         # reglas SAST propias para TypeScript/NestJS
|-- spectral/              # reglas de contrato OpenAPI
|-- templates/
|   `-- microservicio_nestjs/ # plantilla Copier
|-- orchestrator/          # motor en Python
|   |-- cli.py             # Typer
|   |-- api.py             # FastAPI (sirve también el dashboard)
|   |-- llm/               # adaptador de proveedores y prompts
|   |-- knowledge/         # carga a Neo4j y GraphRAG
|   |-- ingest/            # fase 1
|   |-- architecture/     # fase 2
|   |-- generation/       # fase 3
|   |-- gates/             # ejecución de herramientas y conversión a SARIF
|   |-- refinement/       # máquina de estados
|   |-- integration/      # fases 7-8
|   |-- testing/          # fase 9
|   |-- packaging/        # fases 10-11
|   |-- notify/           # Telegram
|   `-- report/           # informe Jinja2 + WeasyPrint
|-- tools/ts-integrity/    # script Node con ts-morph
|-- dashboard/            # React + Vite
|-- evaluation/
|   |-- datasets/         # documentos de requisitos de prueba
|   |-- baselines/       # configuraciones B0, B1, F
|   |-- benchmark_bd/     # comparación Neo4j vs PostgreSQL + pgvector
|   `-- experimentos.py
|-- scripts/instalar_herramientas.sh
|-- docker-compose.dev.yml # Neo4j, Keycloak, RabbitMQ
|-- pyproject.toml
`-- .github/workflows/
```

13. Organización del equipo y hoja de ruta

13.1 Capacidad disponible

El proyecto se desarrolla entre el **1 de octubre de 2026** y el **15 de enero de 2027**, según el calendario de trabajo del equipo. Se consideran las horas indicadas en el calendario como horas *por integrante*.

Concepto	Valor
Días de trabajo	62 (4 horas cada uno)
Días sin trabajo dentro del periodo	9 de octubre; 2 y 3 de noviembre; 20 de noviembre; 7 de diciembre; receso del 19 de diciembre al 3 de enero
Horas por integrante	248
Horas totales del equipo	496
Integrantes	2

13.2 Reparto de responsabilidades

Para trabajar en paralelo sin bloquearse, cada integrante es responsable de una línea de trabajo, y ambos se reúnen en los días de integración y de revisión.

Integrante	Responsabilidad principal
Integrante A: motor e inteligencia	Motor en Python, adaptador de LLM, base de conocimiento y Neo4j, GraphRAG, políticas OPA, fases 1–3 (parte de IA), motor de refinamiento, trazabilidad, informe de evidencia y análisis de la evaluación.
Integrante B: plataforma y seguridad	Microservicio NestJS de referencia y plantilla, CI/CD y notificaciones de Telegram, herramientas de los tres Security Gates, integración (Traefik, RabbitMQ, Keycloak), pruebas, contenedores, DAST y dashboard React.

- **Revisión cruzada:** todo cambio entra a `main` por pull request revisado por el otro integrante. Esto cumple además el control de seguridad sobre la base de conocimiento.
- **Sincronización diaria:** 10 minutos al inicio de cada jornada para revisar el tablero de GitHub Projects y bloqueos.
- **Último día de cada sprint:** integración, demostración, informe del sprint y planificación del siguiente. Este día funciona también como margen si algo se retrasó.

13.3 Resumen de sprints

Tabla 13.1: Sprints ajustados al calendario del equipo.

Sprint	Fechas	Días	Horas equipo	Objetivo e hito verificable
0	1–2 oct	2	16	Planificación. Repositorio creado, herramientas seleccionadas, backlog e informe.
1	5–16 oct	9	72	Walking skeleton y fase 4 inicial. Se construyen las herramientas del Security Gate 1 (tsc , Simgrep, ESLint, Gitleaks, Trivy, Hadolint) y un microservicio NestJS escrito a mano lo pasa en GitHub Actions; sus hallazgos quedan en Neo4j. Proveedor de LLM elegido.
2	19–30 oct	10	80	Conocimiento, OPA y Telegram. Base de conocimiento v1 en Neo4j con GraphRAG; políticas de los tres gates; Gates 2 y 3 iniciales; avisos de cada fase en Telegram; benchmark de bases de datos.
3	4–13 nov	8	64	Fases 1 y 2. Del documento de requisitos al modelo estructurado, la arquitectura validada por OPA, los ADR y el diagrama C4. Dataset de evaluación y primera versión del dashboard.
4	16–27 nov	9	72	Fases 3 y 4. Plantilla NestJS completa; el framework genera microservicios (fase 3) y les aplica el Security Gate 1 completo con decisión de OPA, registro en Neo4j y aviso en Telegram (fase 4). Los bloqueantes encontrados son la entrada del sprint 5.
5	30 nov–11 dic	9	72	Fases 5–8. Motor de refinamiento con todas las salvaguardas; integración con Traefik, RabbitMQ y Keycloak; Gate 2 sobre el sistema generado.
6	14–18 dic	5	40	Fase 9. Pruebas de integración, end-to-end, contrato y autorización. Pipeline de las fases 1–9 funcionando antes del receso.
7	4–8 ene	5	40	Fases 10–11 e informe. Imágenes, SBOM, DAST, Gate 3, informe de evidencia y dashboard final. Framework completo.
8	11–15 ene	5	40	Evaluación DSR y cierre. Experimentos B0, B1 y F; análisis; ajustes; documentación y entrega final.
Total		62	496	

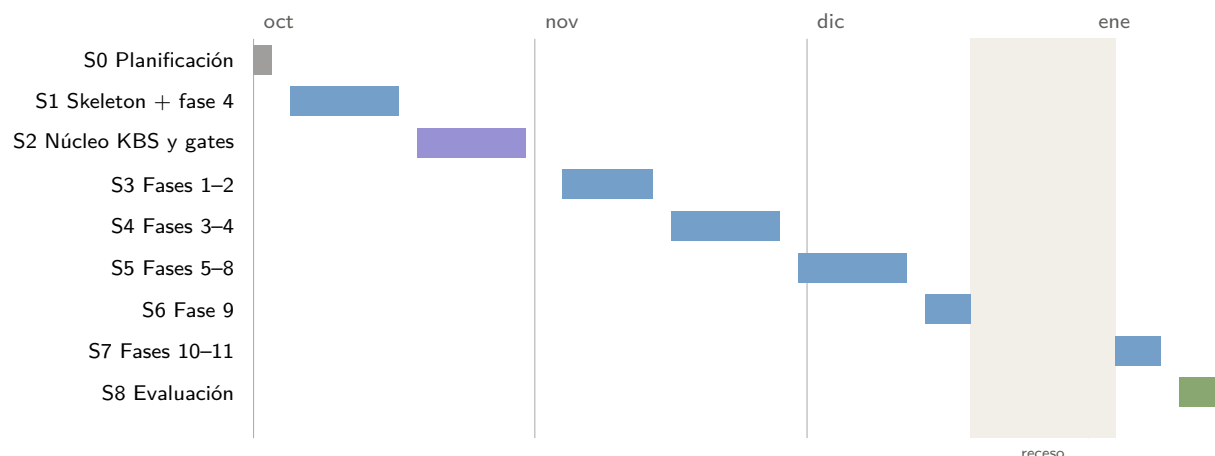


Figura 13.1: Cronograma de sprints entre octubre de 2026 y enero de 2027.

13.4 Cobertura de las fases por sprint

Cada fase del pipeline se construye en dos momentos: una **versión inicial**, sobre el microservicio de referencia escrito a mano, que permite tener la tubería completa desde el primer mes, y una **versión completa**, aplicada al código que genera el framework. La tabla muestra en qué sprint se trabaja cada fase, de modo que ninguna queda sin cubrir.

Fase	Nombre	Versión inicial	Versión completa
1	Adquisición de requisitos	–	Sprint 3
2	Análisis y propuesta arquitectónica	–	Sprint 3
3	Generación de la estructura interna	Sprint 1 (servicio de referencia)	Sprint 4 (plantilla y generación)
4	Security Gate 1: análisis estático	Sprint 1 (herramientas en CI)	Sprint 4 (sobre el código generado, con OPA y Neo4j)
5–6	Refinamiento iterativo	–	Sprint 5 (desde Gate 1); sprint 6 (desde Gates 2 y 3)
7	Integración de microservicios	Sprint 2 (Keycloak en el servicio de referencia)	Sprint 5
8	Security Gate 2: integración	Sprint 2 (Spectral, Conftest)	Sprint 5
9	Pruebas automatizadas	Sprint 1 (Jest básico)	Sprint 6
10	Contenerización	Sprint 1 (Dockerfile); sprint 2 (Trivy image, Syft)	Sprint 7
11	DAST y Security Gate 3	Sprint 2 (ZAP, Nuclei)	Sprint 7
KBS	Base de conocimiento, OPA y GraphRAG	Sprint 2	Sprint 5 (retroalimentación)

–	Notificaciones en Telegram	Sprint 2	Sprint 5 (iteraciones)
–	Dashboard React	Sprint 3	Sprint 7
DSR	Evaluación	Sprint 3 (dataset)	Sprint 8

13.5 Hitos de control

Fecha	Hito
16 oct 2026	Fase 4 inicial: un microservicio escrito a mano pasa el Security Gate 1 en CI. Si no se cumple, no se avanza a la generación con LLM.
30 oct 2026	Walking skeleton completo (tres gates) con avisos en Telegram. Primer hito importante del proyecto.
27 nov 2026	Fases 1–4: el framework genera microservicios desde un documento de requisitos y les aplica el Security Gate 1.
18 dic 2026	Pipeline de las fases 1–9 funcionando antes del receso.
8 ene 2027	Framework completo de las fases 1–11.
15 ene 2027	Evaluación terminada y entrega final.

13.6 Cronograma diario

La tabla siguiente detalla qué hace cada integrante en cada día de trabajo. Las filas de un solo bloque indican trabajo conjunto.

Tabla 13.2: Cronograma diario por integrante.

Fecha	Integrante A (motor e IA)	Integrante B (plataforma y seguridad)	Entregable
Sprint 0: Planificación			
01/10/2026 jue	Ambos: planificación, selección de herramientas y creación del repositorio (estructura, ramas, protección de <code>main</code>).		Repositorio, informe
02/10/2026 vie	Ambos: plan técnico v2, backlog en GitHub Projects, cuentas y claves (LLM, bot de Telegram, secretos).		Plan técnico, backlog
Sprint 1: Walking skeleton y fase 4 inicial (Security Gate 1)			
05/10/2026 lun	Esqueleto del motor: Typers, FastAPI, configuración.	<code>docker-compose.dev</code> : Neo4j, RabbitMQ, Keycloak.	
06/10/2026 mar	Adaptador de LLM con 2–3 proveedores candidatos.	Microservicio NestJS de referencia (usuarios): módulo, DTO, controlador.	
07/10/2026 mié	Prueba comparativa de modelos con 5 requisitos; elección del proveedor.	Prisma + PostgreSQL en el servicio de referencia; Jest básico.	Proveedor de LLM elegido
08/10/2026 jue	Modelo común de hallazgos y lector de SARIF.	Dockerfile multietapa + Hadolint; workflow con un paso por fase.	
12/10/2026 lun	Conexión a Neo4j: restricciones e índices (scripts Cypher).	Gate 1 en CI: <code>tsc</code> , <code>Semgrep</code> , <code>ESLint</code> , <code>Gitleaks</code> , <code>Trivy</code> .	
13/10/2026 mar	Registro de ejecuciones y hallazgos en Neo4j.	Publicación de SARIF en GitHub; <code>npm ci --ignore-scripts</code> .	

Fecha	Integrante A (motor e IA)	Integrante B (plataforma y seguridad)	Entregable
14/10/2026 mié	Comando framework run --fase 4 integrado al CI.	Primeras reglas Semgrep propias para TypeScript/NestJS.	
15/10/2026 jue	Ambos: integración del walking skeleton hasta el Gate 1.		
16/10/2026 vie	Ambos: pruebas, correcciones y revisión del sprint.		Walking skeleton Gate 1; informe sprint 1
Sprint 2: Núcleo de conocimiento, OPA, Telegram y gates iniciales			
19/10/2026 lun	Base de conocimiento: patrones de microservicios en YAML.	Notificador de Telegram: bot y mensaje de progreso editable.	
20/10/2026 mar	Controles ASVS y catálogo CWE / OWASP API Top 10.	Notificaciones en cada paso del workflow y aviso de fallo.	
21/10/2026 mié	Cargador YAML → Neo4j (nodos y relaciones).	Gate 2 inicial: Spectral con reglas OWASP sobre el servicio de referencia.	
22/10/2026 jue	Índice vectorial; fragmentos y embeddings de OWASP/CWE.	Conftest con políticas para Docker Compose.	
23/10/2026 vie	Consulta GraphRAG (vectores + relaciones).	Realm de Keycloak y guard JWT en el servicio de referencia.	
26/10/2026 lun	Políticas Rego de los Gates 1 y 2 con opa test .	Gate 3 inicial: Trivy image y Syft.	
27/10/2026 mar	Política Rego del Gate 3; justificación en cada decisión.	ZAP API scan con límite de tiempo y Nuclei.	
28/10/2026 mié	Prueba comparativa Neo4j vs PostgreSQL + pgvector.	Conversor de ZAP a SARIF.	Anexo benchmark de BD
29/10/2026 jue	Ambos: walking skeleton completo (Gates 1–3 con OPA en CI).		
30/10/2026 vie	Ambos: revisión y demostración con avisos en Telegram.		Skeleton completo; base de conocimiento v1; informe sprint 2
Sprint 3: Fases 1 y 2			
04/11/2026 mié	Docling: extracción de PDF, DOCX y TXT.	Dataset: documento de requisitos 1 (tienda en línea).	
05/11/2026 jue	Normalización y esquema Pydantic Requisito .	Documentos 2 y 3 (gestión académica, citas médicas).	Dataset de evaluación
06/11/2026 vie	Clasificación RF/RNF y etiquetas con LLM.	Dashboard React + Vite: estructura y carga del documento.	
09/11/2026 lun	Requisitos guardados en Neo4j.	Dashboard: progreso por fase con Server-Sent Events.	
10/11/2026 mar	Propuesta de arquitectura con LLM (bounded contexts).	API del motor: endpoints de ejecución y estado.	
11/11/2026 mié	OPA: reglas arquitectónicas e inferencia de patrones.	Generación de ADR en Markdown.	
12/11/2026 jue	Ciclo LLM ↔ OPA (máximo 3 intentos); nodos Decision .	Diagrama C4 en Mermaid.	
13/11/2026 vie	Ambos: prueba de fases 1–2 con los 3 documentos; revisión.		Requisitos, arquitectura y ADR; informe sprint 3
Sprint 4: Fases 3 y 4 (generación NestJS y Security Gate 1)			
16/11/2026 lun	Prompts de generación con contexto GraphRAG.	Plantilla Copier: módulo, entidades Prisma, DTO.	
17/11/2026 mar	Puntos de extensión y relleno de lógica por LLM.	Plantilla: servicio, controlador, Swagger.	

Fecha	Integrante A (motor e IA)	Integrante B (plataforma y seguridad)	Entregable
18/11/2026 mié	Lista de dependencias permitidas y verificación.	Plantilla: guard Keycloak, ValidationPipe, Helmet, errores.	
19/11/2026 jue	Generación de pruebas Jest con LLM.	Plantilla: Dockerfile, Jest base, configuración por entorno.	
23/11/2026 lun	Ambos: generador <code>arquitectura.json</code> → N microservicios.		
24/11/2026 mar	Fase 4: <code>tsc --noEmit</code> como primer bloqueante del Gate 1 y reintento con el LLM.	Fase 4: herramientas del Gate 1 sobre los servicios generados y conversión a SARIF.	
25/11/2026 mié	Fase 4: decisión de OPA (<code>gates.gate1</code>), hallazgos en Neo4j y trazabilidad requisito → componente → archivo → hallazgo.	Fase 4: aviso del Gate 1 en Telegram y ajustes de la plantilla según los hallazgos.	
26/11/2026 jue	Ambos: prueba de las fases 1–4 con los 3 documentos; registro de los bloqueantes del Gate 1 como entrada del sprint 5.		
27/11/2026 vie	Ambos: revisión del sprint.		Microservicios generados con Security Gate 1 aplicado; informe sprint 4
Sprint 5: Fases 5 a 8 (refinamiento, integración y Security Gate 2)			
30/11/2026 lun	Refinamiento: identificar y priorizar.	Fase 7: Docker Compose generado con Traefik.	
01/12/2026 mar	Seleccionar control desde Neo4j.	Rutas, rate limiting y autenticación en Traefik.	
02/12/2026 mié	Refinar: parche del LLM con alcance acotado.	RabbitMQ con el transporte de NestJS.	
03/12/2026 jue	Salvaguardas: <code>tsc</code> , Jest, fingerprints SARIF.	Realm de Keycloak generado por sistema.	
04/12/2026 vie	Integridad con ts-morph; escalado a revisión humana.	Base PostgreSQL por servicio y redes separadas.	
08/12/2026 mar	Iteraciones en Neo4j y en Telegram.	Gate 2 sobre el sistema generado.	
09/12/2026 mié	Retroalimentación: decisiones validadas.	Políticas Conftest adicionales (no root, solo lectura).	
10/12/2026 jue	Ambos: integración de las fases 1–8.		
11/12/2026 vie	Ambos: revisión del sprint.		Refinamiento e integración; informe sprint 5
Sprint 6: Fase 9 (pruebas automatizadas)			
14/12/2026 lun	Refinamiento disparado desde Gates 2 y 3.	Testcontainers (PostgreSQL, RabbitMQ) en las pruebas.	
15/12/2026 mar	Criterios de paso de la fase 9 en OPA.	Pruebas end-to-end con Supertest vía gateway.	
16/12/2026 mié	Pruebas de autorización desde la base de conocimiento.	Schemathesis autenticado con token de Keycloak.	
17/12/2026 jue	Ambos: pipeline de las fases 1–9 con un documento.		
18/12/2026 vie	Ambos: revisión y respaldo antes del receso.		Pipeline fases 1–9; informe sprint 6
Sprint 7: Fases 10 y 11 (contenedores, DAST y Security Gate 3)			
04/01/2027 lun	Informe de evidencia: plantilla Jinja2.	Imágenes finales, Trivy image y Syft en el pipeline.	
05/01/2027 mar	Matriz de trazabilidad con Cypher.	ZAP autenticado y Nuclei sobre el sistema generado.	

Fecha	Integrante A (motor e IA)	Integrante B (plataforma y seguridad)	Entregable
06/01/2027 mié	Métricas por dimensión; PDF con WeasyPrint.	Gate 3 completo y refinamiento desde DAST.	
07/01/2027 jue	Dashboard: resultados, métricas y descarga.	Calibración del DAST con VAmPI y crAPI.	
08/01/2027 vie	Ambos: pipeline completo de las fases 1-11; revisión.		Framework completo; informe sprint 7
Sprint 8: Evaluación y cierre			
11/01/2027 lun	Experimentos B0 (solo LLM).	Experimentos B1 (LLM + gates).	
12/01/2027 mar	Ambos: experimentos F con los 3 documentos y repeticiones.		
13/01/2027 mié	Análisis de resultados y gráficos.	Ajuste de reglas según resultados y reevaluación.	Resultados de evaluación
14/01/2027 jue	Documentación final del framework.	README, guía de uso y video de demostración.	
15/01/2027 vie	Ambos: presentación final y entrega.		Artefacto validado; informe final

13.7 Prioridades y plan de contingencia

El calendario es ajustado: el sprint 6 y los sprints 7 y 8 tienen cinco días cada uno. Si un sprint se retrasa, se recorta alcance en este orden, sin afectar el núcleo de la tesis:

Prioridad	Elementos
Imprescindible	Base de conocimiento en Neo4j, OPA y los tres gates, generación NestJS con plantilla, motor de refinamiento con salvaguardas, trazabilidad, informe de evidencia, experimentos B0 y F.
Importante	Notificaciones en Telegram, GraphRAG, Keycloak, RabbitMQ, Schemathesis, ZAP, calibración con VAmPI, experimento B1.
Deseable	Dashboard React completo (contingencia: dashboard mínimo en Streamlit), Nuclei, Testcontainers, calibración con crAPI, comparación entre varios proveedores de LLM, benchmark de bases de datos.

Regla de avance

No se empieza el sprint 3 sin haber cumplido el hito del 30 de octubre. Tener la tubería funcionando de punta a punta en el primer mes reduce el mayor riesgo del proyecto: descubrir al final que las piezas no encajan.

14. Riesgos del proyecto y mitigación

Riesgo	Impacto	Mitigación
Alcance excesivo para 62 días	Alto	Prioridades definidas; walking skeleton primero; recorte por la tabla de contingencia.
Sprints cortos al final (5 días)	Alto	Las fases 10–11 tienen versión inicial desde el sprint 2; el sprint 7 solo las completa.
Resultados del LLM inconsistentes	Alto	Plantillas deterministas, salida estructurada, <code>tsc</code> , temperatura baja, versión fijada.
Pérdida de contexto tras el receso	Medio	Respaldo, documentación y lista de tareas pendientes el 18 de diciembre.
Dependencia de una sola persona por área	Medio	Revisión cruzada de pull requests; ambos integrantes conocen todo el pipeline.
Tiempos de CI demasiado largos	Medio	Límite de tiempo en ZAP, caché de imágenes y dependencias, ejecución paralela de escáneres.
Límites o costes de la API de LLM	Medio	Caché de respuestas, modelo rápido por defecto, opción de modelo local con Ollama.
Falta de recursos en equipos locales	Medio	Ejecutar la suite completa en GitHub Actions; Keycloak y RabbitMQ solo en fases 7 a 11.
Curva de aprendizaje de Rego y Cypher	Medio	Empezar con políticas y consultas simples; usar el Rego Playground y Neo4j Browser para probar.
Cambios de licencia de herramientas	Bajo	Alternativas abiertas identificadas (Opengrep para Semgrep).

A. Glosario

ADR

Architecture Decision Record: documento breve que registra una decisión arquitectónica, su contexto y sus consecuencias.

Cypher

Lenguaje de consultas de Neo4j.

DAST

Dynamic Application Security Testing: análisis de seguridad de una aplicación en ejecución.

DSR

Design Science Research: metodología de investigación orientada a construir y evaluar artefactos.

GraphRAG

Variante de RAG que recupera, además de fragmentos de texto, los nodos y relaciones conectados en un grafo de conocimiento.

KBS

Knowledge-Based System: sistema que razona a partir de una base de conocimiento explícita.

NestJS

Framework de Node.js para construir servicios backend en TypeScript con arquitectura modular.

OPA

Open Policy Agent: motor de políticas de propósito general que evalúa reglas escritas en Rego.

RAG

Retrieval-Augmented Generation: técnica que añade al prompt información recuperada de una base de conocimiento.

SARIF

Static Analysis Results Interchange Format: estándar OASIS para resultados de análisis.

SAST

Static Application Security Testing: análisis de seguridad del código fuente sin ejecutarlo.

SBOM

Software Bill of Materials: inventario de componentes y dependencias de un software.

SCA

Software Composition Analysis: análisis de vulnerabilidades en dependencias de terceros.

Walking skeleton

Implementación mínima que recorre todo el sistema de punta a punta.